

Chapter 3

Flow Control and Debugging

Normally, code lines in a procedure are sequentially executed from top to bottom. But in many applications, the flow of a program has to be branched based on run-time conditions. Besides branching, some code blocks may need to be executed repeatedly until a certain job is done.

This chapter introduces VB flow control techniques. Specifically, it will introduce branching with If block and Select Case block, as well as repetition with For ... Next and Do ... Loop blocks.

With whatever flow control technique, you must use some conditions to control the branching or repetition. Therefore, before introducing flow control, some related knowledge including the ASCII and relational / logical operations will be discussed.

1. ASCII

You know that computers process information encoded in binary code made up of 0s and 1s stored as charged or discharged state of micro transistors. You may also know that decimal numbers, such as 123, can be easily converted into binary format such as 1111011 (You can try it with a pocket calculator). You may wonder how characters are converted into binary codes.

In fact, every character, whether it appears on the keyboard or not, has a code. The code is the American Standard Code for Information Interchange (ASCII). For example, the ASCII code for letter "A" is 65, letter "a" is 97, and the copyright symbol "©" is 169. For a complete list of all the 256 ASCII characters and symbols, please refer to website <http://www.ascii-code.com/>

VB provides two built-in functions that carry out conversion between characters and ASCII codes:

Asc(c)	– gets the ASCII code of any character (c).
Chr(n)	– converts any number (n) between 0 – 255 to a character.

The following examples show how these functions are used.

Asc("A") returns 65
Chr(169) returns ©



Write a VB 2010 program to convert between characters and ASCII codes. The program uses two text boxes and two buttons. One text box will be used for characters, and the other for ASCII code. One button, when clicked, will convert a character to an ASCII code, and the other button will do the reverse.

Knowing ASCII is important for programming. First, you can use the code to display characters that are unavailable on the keyboard. For example, if you want to display the degree symbol of temperature or to break a line of text in a message box, you have to use ASCII code. For example

```
MessageBox.Show("The temperature is 65" & Chr(176) & "F")
MessageBox.Show("A long message can be displayed" & Chr(10) & _
    "in multiple lines.")
```

The first message box will display "The temperature is 65°F" because ASCII code 176 is the degree symbol, and the second message box will display the long messages in two lines. ASCII code 10 is an invisible line break character.

Second, you can compare strings and sort strings in ascending or descending order. To compare two strings is to compare the ASCII code letter-by-letter in the strings. For example, "A" < "a" (because "A"=65 and "a"=97), "dog" > "cat" (because the first letter "d"=100 and "c"=99 instead of the animal size), and "car" < "cat" (because the third letter "t"=116 and "r"=114 instead of the cat is dearer). Isn't it funny? In the computer, characters are sorted as the binary format of their ASCII code.

2. Relational and Logical Operations

In computer programming, relational and logical operations or expressions are often conditions used to control program flows. A relational operation uses a relational operator to compare two values for equality / inequality. In VB, relational operators include.

=	equal to
<>	not equal to
<	less than
>	greater than
<=	less than or equal to
>=	greater than or equal to

The result of a relational operation or expression is a Boolean value. Boolean is a data type in VB. A Boolean value can either be True or False. You can declare variables of Boolean type, and assign Boolean values to Boolean variables. The following are a few examples of relational operations, each resulting a Boolean value.

Relational operation	Result
(1 < 2)	True
"cat" > "car"	True
"AM" > "am"	False

The following example shows a Boolean variable (*bool*) is declares and is assigned a value in the form of a relational operation. Since the relational operation is evaluated False, the value carried by variable *bool* is thus False.

```
Dim bool as Boolean = "cat" > "dog"
```

A logical operation uses a logical operator (And, Or, Not) to combine two conditions (Boolean values or relational operations). The result of a logical operation is also a Boolean value.

And binary operator used to combine two conditions
 Or binary operator used to combine two conditions
 Not unary operator used to reverse one condition

Since a relational operator can only compare one value with another, to represent some commonly used math expressions, such as $0 < x < 10$, in which one variable (x, carrying one value) is simultaneously compared with other two values (0 and 10), relational operators need to be used to create complex logical expressions, e.g. $x > 0$ And $x < 10$. The following are a few examples of relational and logical expressions.

Given	Expression	Result
n = 3	(n > 1) And (n < 5)	True
n = 6	(n > 1) And (n < 5)	False
n = 6	(n > 1) Or (n < 5)	True
n = 3	Not (n < 1)	True
n = 3	Not (n > 1)	False

The following lists all possible logical operations and results.

Condition 1	Operator	Condition 2	Result
True	And	True	True
False	And	False	False
True	And	False	False
False	And	True	False
True	Or	True	True
True	Or	False	True
False	Or	True	True
False	Or	False	False
	Not	True	False
	Not	False	True

In summary, when both conditions are true the result of an And operation is True; when any condition is false, the result of an And operation is False; when any condition is true, the result of an Or operation is True; when both conditions are False, the result of an Or operation is False. The unary operator Not reverses the condition that follows it.

3. Decision

During the execution of a program, the program flow often needs to be switched to different tracks to handle different conditions. In VB, switching tracks for the program flow is carried out by the If block or Select Case block or their variations.

3.1 If block

Syntax 1: Basic If ... End If block

```
If <condition> Then
    (do something)
End If
```

The opening statement “If <condition> Then” evaluates a condition. The condition can be a relational / logical expression that can produce a result. Or it can be a Boolean variable carrying a definite value. If the condition is true, the program executes the code inside the block. Otherwise, the program flow skips to the closing statement “End If”. Please do not include the angle brackets in your programs (see Chapter 2 Section 1).

Syntax 2: If ... Else ... End If block

```
If <condition> Then
    (do action 1)
Else
    (do action 2)
End If
```

The opening statement “If <condition> Then” evaluates the condition. If the condition is true, it executes the code for action 1. Otherwise, it executes the code for action 2 in the “Else” block.

Syntax 3: If block nesting

```
If <condition 1> Then
    If <condition 2> Then
        (action 1)
    Else
        (action 2)
    End If
Else
    If <condition 3> Then
        (action 3)
    Else
        (action 4)
    End If
End If
```

In this nested If block, the program evaluates conditions from top to bottom and from high-level to low-level sub-blocks. If the condition of a sub-block is true, that sub-block is executed. After executing code in the sub-block, the program jumps to the last closing statement “End If” without evaluating any remaining condition. Theoretically, there is no limit in the number of levels if blocks can nest, but in the entire hierarchical structure, only one action is executed.

Syntax 4: Elself block

```
If <condition 1> Then
    (do action 1 if condition 1 = true)
ElseIf <condition 2> Then
    (do action 2 if condition 2 = true)
```

```

ElseIf <condition 3> Then
    (do action 3 if condition 3 = true)
Else
    (do action 4 for all other conditions)
End If

```

Using “ElseIf” allows an If block to evaluate multiple conditions. With this structure, the program sequentially evaluates each condition from top to bottom. Once a condition is evaluated True, the program flows into that sub-block. After executing the code in that sub-block, the program jumps to the closing statement “End If” by ignoring all rest conditions. If the conditions are mutually exclusive, the order in them does not matter. However, if the conditions are not mutually exclusive, they must be properly ordered so that a condition does not accommodate any condition below it. For example,



```

If ages <= 3 Then
    ' display go to day care
ElseIf age <= 6 Then
    ' display go to pre-school
ElseIf age <= 12 Then
    ' display go to elementary school
ElseIf age <= 14 Then
    ' display go to middle school
ElseIf age <= 18 then
    ' display go to high school
Else
    ' display go to college
End If

```

In this sequential condition structure, an earlier condition does not accommodate any latter one. For example, “age <= 3” does not accommodate “age <=6”, and “age <= 6” does not accommodate any condition behind it. But it is fine for a later condition to accommodate an earlier one. For example, it is OK for condition “age <= 6” to include condition “age <= 3”, because when this condition is evaluated it is not possible for variable *age* to be less than or equal to 3 (which has already been handled by the previous sub-block). Therefore, what is handled by the sub-block with condition “age <= 6” is actually age between 3 and 6 (including 6). Think what would happen if the conditions are rearranged in the reverse order as the following:

```

If ages <= 18 Then
    ' display go to high school
ElseIf age <= 14 Then
    ' display go to middle school
ElseIf age <= 12 Then
    ' display go to elementary school
ElseIf age <= 6 Then
    ' display go to pre-school
ElseIf age <= 3 then
    ' display go to day care
Else
    ' display go to college
End If

```

To check validity of the above code, please use an age value, say 10, to test it.



Example 1: Water stage

A lake is fed by a river and discharges through a dam. If the discharge (m^3/s) is equal to the in-flow (m^3/s), the water stage of the lake stays unchanged. If the discharge is greater than in-flow, the water stage will fall; otherwise, water stage will rise. Write a program that takes two user input values, discharge and in-flow, and determines the change in water stage of the lake.

First, create a new VB project from the Windows forms application template, name the project “WaterStage”, and design the project GUI as Figure 1.

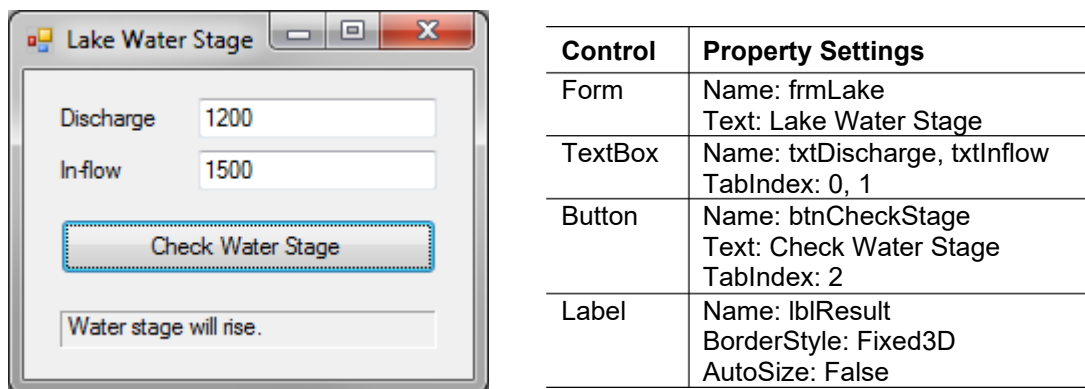


Figure 1. GUI of the water stage project

Then, implement the button click event procedure using the following code.

```
Private Sub btnCheckStage_Click(...) Handles btnCheckStage.Click
    Dim dischg As Integer = CInt(txtDischarge.Text)
    Dim inflow As Integer = CInt(txtInflow.Text)

    If dischg = inflow Then
        lblResult.Text = "Water stage will stay unchanged."
    Else
        If dischg > inflow Then
            lblResult.Text = "Water stage will fall."
        Else
            lblResult.Text = "Water stage will rise."
        End If
    End If
End Sub
```

By using an “Elsif” statement in the If block, the following code does the same job.

```
Private Sub btnCheckStage_Click(...) Handles btnCheckStage.Click
    Dim dischg As Integer = CInt(txtDischarge.Text)
    Dim inflow As Integer = CInt(txtInflow.Text)

    If dischg = inflow Then
        lblResult.Text = "Water stage will stay unchanged."
    Elself dischg > inflow Then
        lblResult.Text = "Water stage will fall."
    Elself dischg < inflow Then
        lblResult.Text = "Water stage will rise."
    End If
End Sub
```

```

ElseIf dischg > inflow Then
    lblResult.Text = "Water stage will fall."
Else
    lblResult.Text = "Water stage will rise."
End If
End Sub

```

The solution can be explained by the following flowchart (Figure 2).

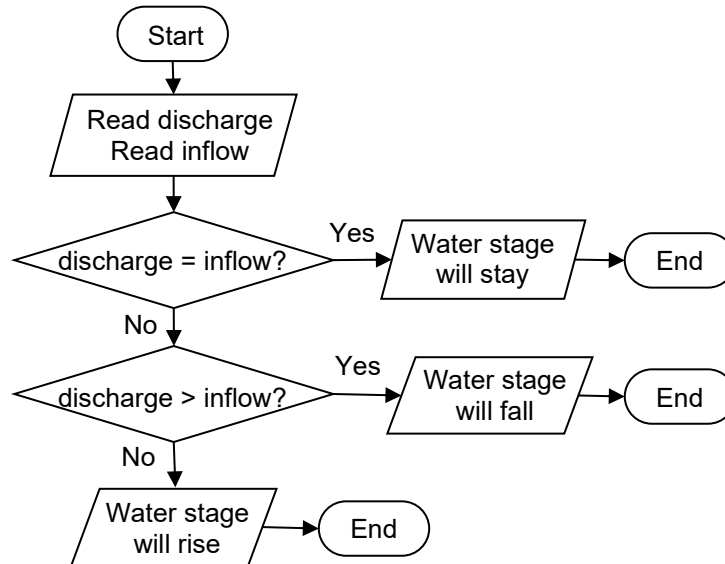


Figure 2 Flowchart of the water stage problem

3.2 Select Case block

Syntax 4 of the If block branches program flow into multiple sub-blocks based on a series of conditions. If the program flow is diverted based on various results of one expression or variable, it can be more convenient to use the Select Case block.

Syntax:

```

Select Case <a variable or expression>
    Case <value1>
        (action 1 if variable = value1)
    Case <value2>
        (action 2 if variable = value2)
    Case Else
        (action n for other values)
End Select

```

In the Select Case block, a case variable or expression is used to control the program flow. value1, value2 and so on are different values of the case variable or results of the expression. The opening statement "Select Case" evaluates the value of the case variable or the expression, if the result matches any case value below, the corresponding sub-block is executed. If none of the listed case values match the case variable value or case expression value, the "Case Else" sub-block is executed. In one

execution of the case block, only one case sub-block will be executed. After executing a sub-block, the program flows to the closing statement “End Select”.



Example 2: Lunch menu

Write a program to display the weekly lunch menus of an elementary school. Use a text box (txtInput) for the user to enter a day (Monday to Sunday), another box (txtOutput) to display the menu of the day, and a button (btnFind) to execute query. The following is the button’s Click event procedure.

```
Private Sub btnFind_Click(...) Handles btnFind.Click
    Select Case txtInput.Text.ToUpper()
        Case "MONDAY"
            txtOutput.Text = "Grilled hamburger"
        Case "TUESDAY"
            txtOutput.Text = "Chicken with mashed potatoes"
        Case "WEDNESDAY"
            txtOutput.Text = "Cheese pizza"
        Case "THURSDAY"
            txtOutput.Text = "Fish, rib or hamburger patty on bun"
        Case "FRIDAY"
            txtOutput.Text = "Macaroni and cheese"
        Case "SATURDAY", "SUNDAY"
            txtOutput.Text = "No school, no lunch."
        Case Else
            txtOutput.Text = "Check spelling of your input"
    End Select
End Sub
```

In this example, the case expression is `txtInput.Text.ToUpper()` which is the content of the input text box (i.e. the Text property value of the txtInput control). `ToUpper()` is a method of the string object that converts the string into all upper case letters. Please note that all case values (“MONDAY”, “TUESDAY”...) are capitalized. The purpose of converting the user input to upper case letters and using all capitalized case values is to make the user input case insensitive in this select case block.



If the case variable is a discrete numeric type such as integer, ranges can be used in case values. For example, the following is the button click event that finds student grade based on a user input (in a text box named txtInput).

```
Private Sub btnFind_Click(...) Handles btnFind.Click
    Dim Age As Integer = CInt(txtInput.Text)
    Select Case Age
        Case Is < 3
            txtOutput.Text = "Day care"
        Case 3 To 5
            txtOutput.Text = "Kindergartener"
        Case 6 To 11
            txtOutput.Text = "Elementary school student"
        Case 12 To 13
            txtOutput.Text = "Middle school student"
        Case 14 To 17
            txtOutput.Text = "Highschool student"
    End Select
End Sub
```

```

        Case Is > 18
            txtOutput.Text = "College Student"
        End Select
    End Sub

```

If the case variable is a continuous numeric type such as Single or Double, case values can be expressed in another way:

```

Private Sub btnFind_Click(...) Handles btnFind.Click
    Dim temperature As Single = CSng(txtInput.Text)
    Select Case temperature
        Case Is <= 40
            txtOutput.Text = "Frigid"
        Case Is <= 59
            txtOutput.Text = "Cold"
        Case Is <= 68
            txtOutput.Text = "Cool"
        Case Is <= 80
            txtOutput.Text = "Comfortable"
        Case Else <= 95
            txtOutput.Text = "Hot"
        Case Else
            txtOutput.Text = "Blazing"
    End Select
End Sub

```

4. Repetition

Repetitions are also called loops. VB.NET provides three basic repetition structures. If the times to repeatedly execute an action is known or fixed, the “For ... Next” repetition can be used. If the number of iterations is unknown or cannot be expected because it depends on run-time conditions, the “Do ... Loop” or “While ... End While” repetition can be used.

4.1 For ... Next repetition

Syntax

```

Dim i As Integer
For i = m to n [Step s]
    (action ...)
Next

```

This repetition block is opened by the For statement and closed by the Next statement. The block uses a numeric variable (*i*) to control the number of iterations. The For statement starts the repetition by assigning an initial value (*m*) to the control variable (*i*) and terminates the repetition when *i* exceeds the upper bound (*n*). The optional Step statement defines that *i* will be incremented by *s* after each round of iteration. If the increment (*s*) is 1 the Step statement can be omitted. Inside the block is the action code to execute. After executing the action code once, the program flows to the Next statement where the control variable (*i*) is incremented by *s*. If the control variable (*i*) value is still between *m* and *n* (inclusive) after incrementation, the program flows back to

the top line of the action code to execute the action again. Otherwise, the repetition terminates and the program flows to any code that follows the block.

If the iteration control variable *i* is used only inside this repetition structure, i.e. if *i* will not be used by any code after the repetition block, it can be declared in the opening line as the following:

```
For i As Integer = m to n [step s]
    (action ...)
Next
```



For example,

```
For year As Integer = 2000 To 2010 Step 2
    lstBox.Items.Add(year)
Next
MessageBox.Show("Number of years in the box: " & lstBox.Items.Count)
```

The repetition in this example adds some year numbers into a list box. Once the repetition starts, a control variable *year* is created and assigned an initial value 2000. Then the program flows into the action block, i.e. the line that adds the *year* value (2000) into the list box. After executing the action, the program hits the closing statement *Next* where variable *year* is incremented by 2 to become 2002. Since 2002 is between 2000 and 2012, the program flows back to the action code again, and value 2002 is added into the list box. The program keeps repeating in this way until the control variable *year* becomes 2012. Since 2012 exceeds the termination value 2010, the repetition stops and the program flows to any code following the repetition block. Finally, a message box pops up to display “Number of years in the box: 6”.



Example 3: Banking

Banks calculate interest monthly. If you deposit a sum of money in a savings account, every statement period (month) the amount is increased by the current balance multiplying a monthly interest rate. Create a VB program to compute account balance of a specified period (months) with a given annual percentage rate (APR).

First, create a new VB 2010 project named “Banking” and use the following property settings when designing the project GUI (Figure 3).

Control	Property Settings
Form	Name: frmBanking Text: Banking
TextBox	Name: txtInitial TabIndex: 0
TextBox	Name: txtAPR TabIndex: 1
TextBox	Name: txtMonths TabIndex: 2
Button	Name: btnCompute TabIndex: 3 Text: Compute
ListBox	Name: lstReport

Figure 3. GUI of the banking project

Then use the following code to implement the compute button's Click event procedure. Please also pay attention to the comments with the code.

```

Private Sub btnCompute_Click(...) Handles btnCompute.Click
    ' validates user input
    If txtInitial.Text = "" Then Return
    If txtAPR.Text = "" Then Return
    If txtMonths.Text = "" Then Return

    ' gets user inputs and store them in variables
    Dim bal As Single = CSng(txtInitial.Text)
    Dim apr As Single = CSng(txtAPR.Text) / 100 ' annual percentage rate
    Dim mon As Integer = CInt(txtMonths.Text) ' number of months

    Dim mpr As Single = apr / 12 ' converts apr to monthly percentage rate

    ' computes monthly balances of the period
    lstReport.Items.Clear() ' clear the list box
    For i As Integer = 1 To mon
        bal += bal * mpr ' same as bal = bal + bal * mpr
        lstReport.Items.Add("Month " & CStr(i) & ": " & FormatCurrency(bal))
    Next
End Sub

```



The first three lines of code are used to validate user input. If any of the text boxes is blank, the procedure terminates. Statement Return skips all the rest code in the procedure and forces the program to flow to the closing statement End Sub.

Please also note that these If blocks have no closing statements End If. If an If block only handles one condition, i.e. there is no Else or Elseif statements, and the action has only one line of code, then the whole If block can be written in one line without using the closing statement. In all these If blocks, the actions are statement Return.

FormatCurrency() is a VB built-in function that converts a number to the currency format.



Example 4: population growth

The population of the City of Flagstaff was 60,000 (in programs do not use commas in numbers) in 2010. Suppose the population keeps growing at a rate of 5% every 5 years for the next 20 years, compute the population of the city during the 20 period at five year interval and display results in a list box. The calculation is very similar to the banking example.

```
Private Sub btnCompute_Click(...) Handles btnCompute.Click
    Dim pop As Long = 600000 ' gets initial population of 2010

    lstReport.Items.Clear() ' clear the list box
    lstReport.Items.Add("2010 population: " & pop.ToString())
    lstReport.Items.Add("Population growth rate: 5% per 5 years")

    For i As Integer = 2010 To 2030 Step 5
        pop += pop * 0.05 ' same as pop = pop + pop * 0.05
        lstReport.Items.Add("Year " & CStr(i) & ": " & CStr(pop))
    Next
End Sub
```

4.2 Nested repetitions

A For ... Next repetition can be nested in another repetition block. For example,

```
For i = m to n
    [optional action 1 ...]
    For j = p to q
        <main action ...>
    Next
    [optional action 2 ...]
Next
```

In this nested structure, there are two repetitions controlled by variables i and j respectively. Variable i controls the outer loop with a value range between m and n . Variable j controls the inner loop with a value range between p and q . Once the block starts to execute, the outer loop is initialized with variable i assigned with value m . Then optional action 1 is executed if it exists. When the program flows to the inner loop, variable j is assigned an initial value p . After executing the main action, the closing statement Next of the inner loop is reached, where variable j is incremented. The main action is repeated as long as variable j does not exceed value q . Once the inner repetition finishes, optional action 2 is executed if it exists. Then the closing statement Next of the out loop is reached and variable i is incremented. If variable i does not exceeds n , program flows back to optional action 1. In each round of iteration of the outer loop, the inner loop is initialized so that the value of variable j is brought back to p . These nested repetitions continue to iterate until variable i exceeds value n .



Example 5: Times Table

Given numbers 1, 2, and 3, we can use a nested For ... Next repetition block to compute a multiplication table. Please create a new VB 2010 project named "TimesTable" and design the project GUI as below. The main form contains a button (btnCompute) and a list box (lstTable). When the button is clicked, a resulting multiplication table displays in the list box (Figure 4).

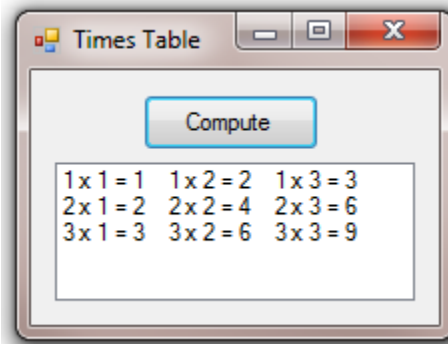


Figure 4 Running result of the times table program

```
Private Sub btnCompute_Click(...) Handles btnCompute.Click
    Dim product As Integer
    Dim rowstr As String

    For i As Integer = 1 To 3
        rowstr = "" ' resets variable rowstr
        For j As Integer = 1 To 3
            product = i * j ' calculates i * j
            rowstr &= i & " x " & j & " = " & product & "    "
        Next
        lstTable.Items.Add(rowstr) ' displays a row
    Next
End Sub
```

4.3 Do ... Loop and While ... End While

These two repetition blocks are used when the number of iterations is initially unknown or cannot be expected, and the termination of the repetition is determined by run-time conditions. For example, a game generates a random number between 0 and 9, and it lets you to guess the number by typing an answer in a text box. If your answer does not match the number, it clears the text box and sends a message and lets you try again. If you succeed, then it pops up a congratulation message and stops the game. We need to use a repetition for this problem. But when we design the program, we do not know the number of repetitions because it depends on your luck in every play.

With these two loop structures, a condition must be used to control the repetition, and the condition must be changed by the action in the loop so that the loop has a chance to terminate.

Syntax 1:

```
Do While <condition>
    (action ...)
Loop
```

Inside this repetition block, the action code is repeatedly executed as long as the condition stays true. Every time the action code is executed, the condition that controls the loop must be changed. Once the condition is turned false by the action, the loop finishes.

Syntax 2:

```
While <condition>
    (action ...)
End While
```

Syntax 2 works exactly same as syntax 1. The only difference is that it uses different keywords While ... End While. This is a newer repetition block introduced to VB.NET.

Syntax 3:

```
Do
    (action ...)
Loop Until <condition>
```

This repetition block is similar to syntax 1. But the loop repeats as long as the condition stays false. Every time the action code is executed, the condition that controls the loop must be changed. Once the condition turns true, the loop finishes.

Syntax 4:

```
Do
    (action ...)
    If <condition> Then
        Exit Do
    End if
Loop
```

Syntax 4 is straightforward. It does not use a condition on the Do or Loop statement. Instead, it uses an embedded If block to terminate the repetition. In each iteration, the action must change the condition, so that when the condition turns true, the loop terminates. Statement Exit Do causes the program flow to jump out of the loop structure.

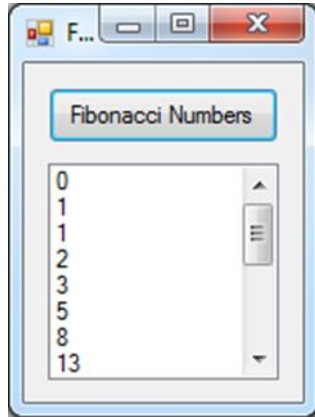


Important: You must design a condition that can be changed by the action in order to terminate a loop! Otherwise, a loop is an infinite (endless) loop. If a program contains an infinite loop, the computer will crash when the program is executed.



Example 6: Fibonacci numbers

Fibonacci numbers is an infinite sequence of natural numbers in which a number is the sum of its two immediate predecessors: 0, 1, 1, 2, 3, 5, 8, 13, ... Let's write a program to generate Fibonacci numbers capped by 1000. We will use various Do ... Loop and While ... End While loop blocks to demonstrate the same job carried out by different repetition blocks. To start, please create a new VB 2010 project named "Fibonacci" with the following project GUI design (Figure 5):



Control	Property Settings
Form	Name: frmFibonacci Text: Fibonacci
Button	Name: btnFibonacci Text: Fibonacci Numbers
ListBox	Name: lstOutput

Figure 5 User interface of the Fibonacci numbers program

To develop a strategy for the task, we can use three variables a , b , and f to represent three consecutive numbers: a as the second predecessor, b as the first predecessor, and f the current Fibonacci number. Initially, $a = 0$ and $b = 1$, so that we can calculate f by $f = a + b$. If f does not exceed 1000, we can display f , let $a = b$ and $b = f$, and then calculate the next f . We can continue to repeat the process until f turns greater than 1000 (Figure 6).

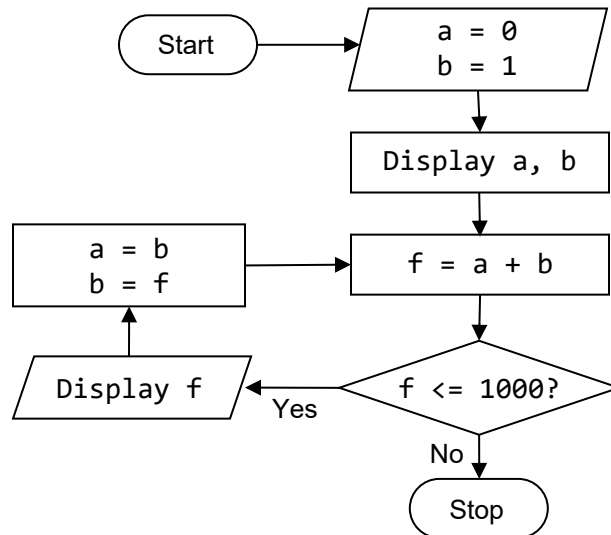


Figure 6 Flowchart for generating Fibonacci numbers

The button click event procedure can be implemented by the following code.

```

Private Sub btnFibonacci_Click(...) Handles btnFibonacci.Click
    Dim a As Integer = 0      ' declares variable a with initial value 0
    Dim b As Integer = 1      ' declares variable b with initial value 1
    Dim f As Integer = 0      ' declares variable f for Fibonacci number

    lstOutput.Items.Clear()   ' clears the list box
    lstOutput.Items.Add(a)    ' adds value of a into the list box
    lstOutput.Items.Add(b)    ' adds value of b into the list box
  
```

```

f = a + b                ' calculates a Fibonacci number

Do While f <= 1000        ' repeats if f does not exceed 1000
    lstOutput.Items.Add(f) ' adds f into the list box
    f = a + b             ' calculates a new fib number
    a = b                 ' pushes value of b to variable a
    b = f                 ' pushes value of f to variable b
Loop
End Sub

```

The repetition is controlled by the While condition in the above code. This repetition may be written using syntax 2 as the following:

```

While f <= 1000          ' repeats if f does not exceed 1000
    lstOutput.Items.Add(f) ' adds f into the list box
    f = a + b             ' calculates a new fib number
    a = b                 ' pushes value of b to a
    b = f                 ' pushes value of f to b
End While

```

Alternatively, the repetition may be written by using syntax 3 with an Until condition as below. Please pay attention to the condition ($f > 1000$). This Until condition is different from the While condition.

```

Do                      ' starts a repetition
    lstOutput.Items.Add(f) ' adds f into the list box
    f = a + b             ' calculates a new fib number
    a = b                 ' pushes value of b to a
    b = f                 ' pushes value of f to b
Loop Until f > 1000      ' terminates when f becomes > 1000

```

Moreover, the repetition may also be controlled by an If condition inside it by using syntax 4:

```

Do
    lstOutput.Items.Add(f)
    f = a + b
    If f > 1000 Then
        Exit Do          ' to jump out of the loop structure
    End If
    a = b
    b = f
Loop

```

Like literature, these repetition blocks add color to programming language while they carry out the same task.

5. Debugging

As your program grows in size and becomes more and more complicated, making errors at times is almost unavoidable. Although VB is smart enough to capture all syntax errors, and can report run-time errors (by crashing), you are responsible for logical

errors. If you find a result produced by a program does not make sense or does not match what you expect, you will need to examine the code line by line to find where computation goes wrong. If your computer crashes because of an infinite loop (run-time error), which usually occurs a moment after the program is started, you will need to check your loop controlling conditions and find out why they do not terminate loops.

Thanks to Microsoft for creating such a wonderful programming language and the IDE. There is no other programming language in the world that has such a great debugging tool set as Visual Basic!

VB is the first and by far the best programming language that allows programmers to run a program line by line in debugging. It allows you to set breakpoints in your code to stop the program flow when the program runs. It allows you to manually step through your program line by line, and lets you check variable values and watch them changing while code lines are executed step by step. These debugging capabilities and tools are extremely helpful when you debug branches, loops, and algorithms. In addition, the VB debugging tools can also help learners understand complex program flows.

5.1 Breakpoint

You can simply click on the left border of the code editor to set breakpoints in your code (Figure 7). Once a breakpoint is set, a brown round button appears on the clicking spot on the border, and the code line is highlighted with a brown background. To remove a breakpoint, you just need to click once on the breakpoint.

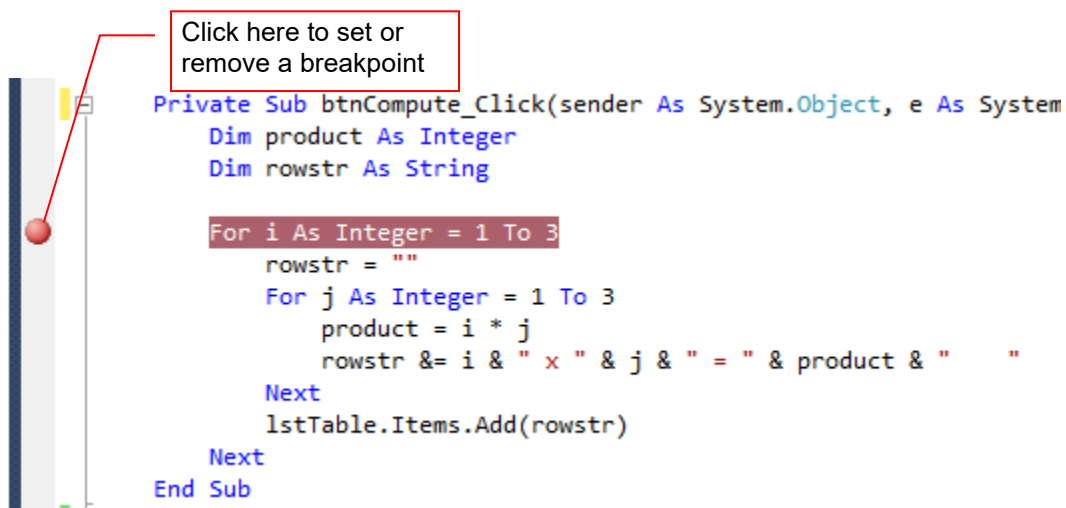


Figure 7 Setting and removing breakpoints

5.2 Stepping through a program

With one or more breakpoints set, when you run (debug) the program in the VB IDE, the program flow stops at the first point where the code line is highlighted with a bright yellow background (Figure 8).

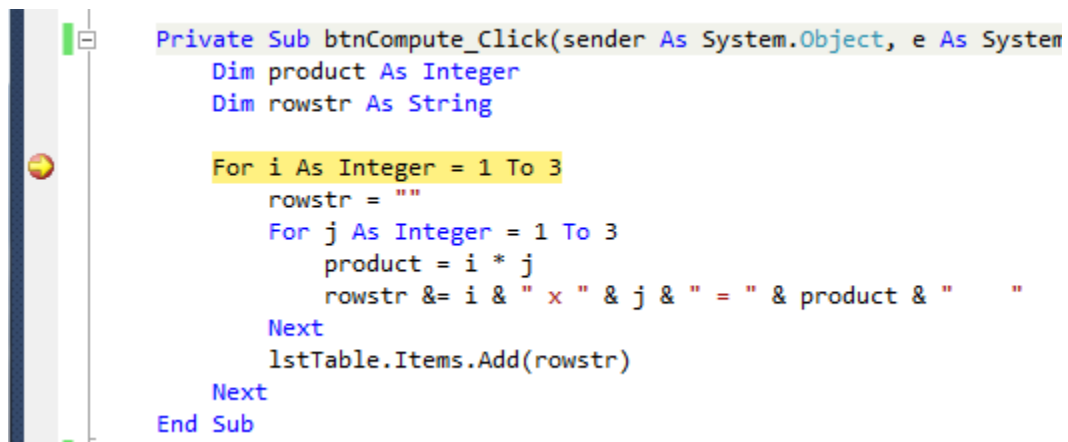


Figure 8 The program flow stops at a breakpoint

Then you can use the F8 key to move a step forward. To watch how the program flows in a procedure, you can repeatedly push the F8 key.

When a program flow stops at a line, you can push the F5 key or click the run (debug) button in the main toolbar to let the program to run to the next breakpoint if any or to through if there are no more breakpoints ahead.

5.3 Examining variable values

There are different ways to check variable values during debugging. The simplest way is to hover your mouse cursor over a variable when the program stops a line during debugging, then the variable value will pop up in a box near the mouse cursor (Figure 9).

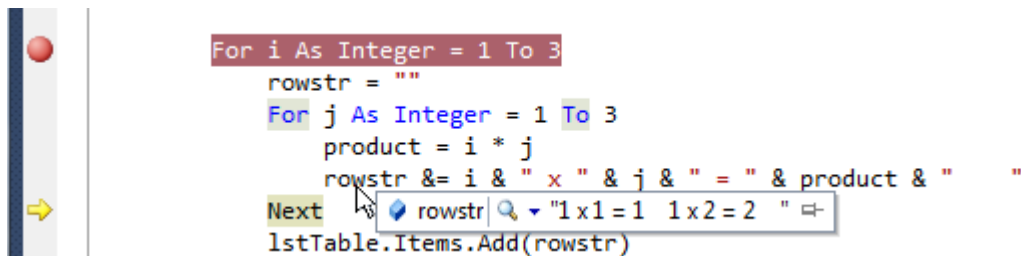


Figure 9 Hover the mouse cursor over a variable to view its current value

While in debugging (running) mode, another easy way is to turn on the Watch window by clicking the main menu: Debug >> Windows >> Watch >> Watch 1 or any. Then in the code editor window, right click on a variable and select "Add Watch". You can add as many variables into the watch window as need. When you use F8 to step through the program, you can observe variable values changing in the watch window (Figure 10).

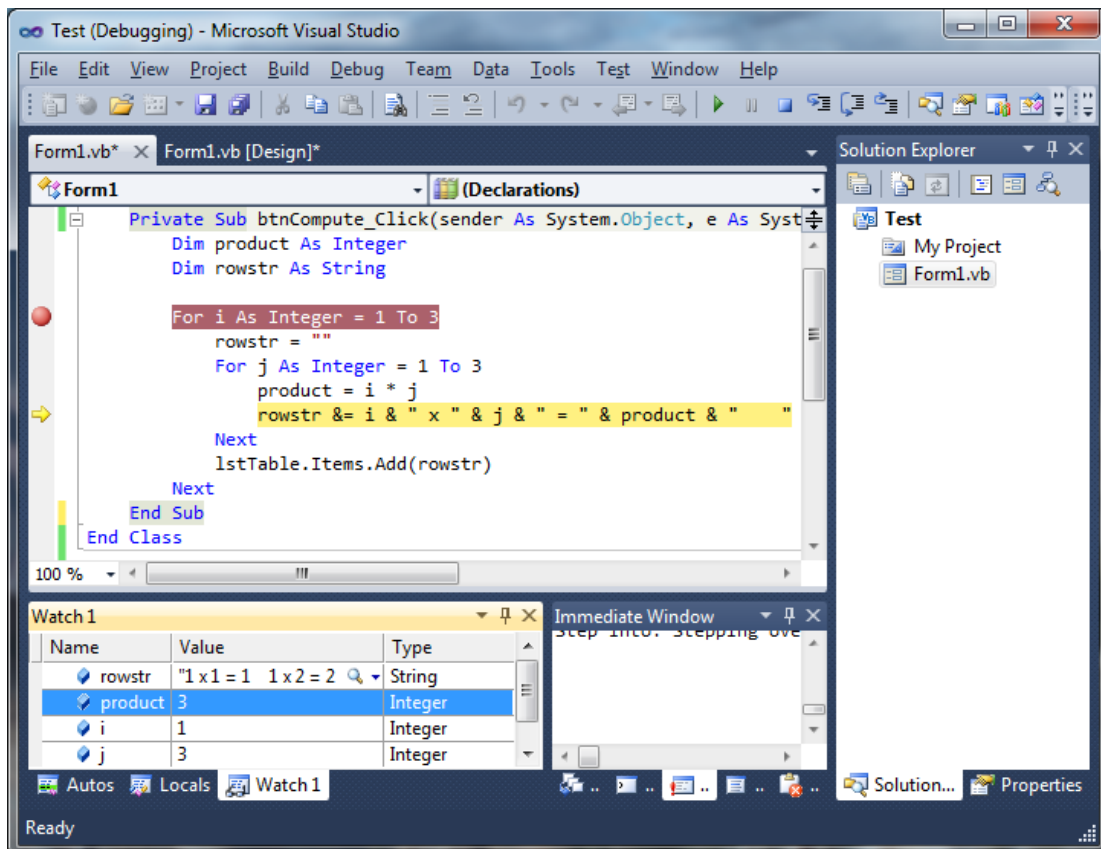


Figure 10 Monitoring variable values in a watch window

Please practice these debugging techniques with the programs you created in this chapter. Use the F8 key to step through the Fibonacci program and learn how the termination condition controls the program flow in a repetition block.

Summary and Highlights

Each character has a ASCII code, so that characters can be converted into binary codes and processed by the computer. The VB built-in function for converting a character to ASCII code is Asc() and the function to convert a ASCII code to character is Chr().

In computer programming, the program flows are often controlled by conditions expressed by relational and logical operations. A relational operation uses a relational operator to compare two values in equality or inequality. The result of a relational operation is Boolean value which can either be True or False. Simple relational operations can be integrated by logical operators (And, Or) to form complex expressions. A unary logical operator Not reverse the value that follows it.

Program flows can be branched by the If block and Select Case block with variations to handle multiple conditions. In an If or Select Case block, once a condition is evaluated as true, the program executes the action under that condition and then exit the block.

Actions can be repeated by using repetition blocks (loops). In VB, three basic repetitions are the For ... Next block, Do ... Loop block, and While ... End While block. The For ... Next block is used when the number of repetitions is known or can be determined by the program, and the Do ... Loop block and While ... End While block are used to handle situations in which the number of repetitions is determined by run-time conditions. For these loops, it is important to make sure that the condition used to control the repetition can be change in each iteration so that it can terminate the repetition.

The VB IDE provides highly sophisticated and convenient debugging tools. You can set a breakpoint at any code line to stop the program flow in debugging mode to examine variable values. When the program flow stops during debugging, you can check variable values by either hovering the mouse cursor over a variable or adding interested variables into a watch window. You can use the shortcut key F8 to move the program by one step, and use F5 to run to the next breakpoint if exists or to the end of the program.

Exercises

1. Answer the following questions.
 - 1) What is ASCII?
 - 2) How do you convert between ASCII codes and characters?
 - 3) What is the Asc() function?
 - 4) What is the Chr() function?
 - 5) What is relational operation?
 - 6) What is logical operation?
 - 7) What are relational operators?
 - 8) What are logical operators?
 - 9) What are results of relational and relational operations?
 - 10) How are strings compared in a relational operation?
 - 11) Are you able to tell results of relational and logical expressions?
 - 12) What is a Boolean variable?
 - 13) How does the program flow in an If block?
 - 14) How many variations are there with the structure of If block?
 - 15) How do you use If blocks in programming?
 - 16) How does the program flow within a Select Case block?
 - 17) How do you use Select Case block in programming?
 - 18) How does the program flow within a For ... Next repetition?
 - 19) How does the For ... Next block control repetitions?
 - 20) What is the repetition (loop) control variable in a For ... Next block?
 - 21) What does keyword "Step" do in a For ... Next block?
 - 22) What does keyword "Next" do in a For ... Next block?
 - 23) What define the lower/upper bounds of the repetition control variable in For ... Next?
 - 24) Can you use the For ... Next repetition in programming?
 - 25) How does the program flow within a Do ... Loop repetition block?
 - 26) What is an infinite loop?
 - 27) How do you control the Do ... Loop repetition so that it does not become an infinite (dead) loop?
 - 28) What does keyword While or Until do in a Do ... Loop block?
 - 29) How do you debug a program?
 - 30) How do you set a breakpoint in your code?
 - 31) How do you remove a breakpoint in your code?
 - 32) How do you step through your code?
 - 33) How do you examine variable values in your code?
2. find results of the following relational and logical expressions
Given $x = 5$, $y = 2$
 - 1) $(x > y) \text{ Or } (x < y)$
 - 2) $(x > y) \text{ And } (x < y * 2)$
 - 3) $(x^2 - y^2) \geq (x - y)^2$
 - 4) $(x + y) \geq (x - y) \text{ And } (x - y * 2 < y) \text{ Or } (y * 3 < x / y)$
 - 5) $x * y \leq x^y \text{ And } (y^x \geq y * x \text{ And } x / y > x - y)$
Given $\text{str1} = \text{"Dog"}$, $\text{str2} = \text{"Dot"}$
 - 6) $\text{str1} > \text{str2}$
 - 7) $\text{str1.Length} \geq \text{str2.Length}$

8) str.ToUpper() < str.ToLower()

Given str = "Visual Studio 2010"

9) str.IndexOf("s") < str.LastIndex("o")

10) str.Substring(7, 6) > str.Substring(7)

3. Analyze the following VB programs and tell execution results in the list box.

1) Suppose text box txtNum1 contains "8" and text box txtNum2 contains "10"

```
Private Sub btnStart_Click(...) Handles btnStart.Click
    Dim num1 As Integer = CInt(txtNum1.Text)
    Dim num2 As Integer = CInt(txtNum2.Text)
    If num1 = num2 Then
        lstOutput.Items.Add("Number 1 and number 2 are equal")
    ElseIf num1 > num2 Then
        lstOutput.Items.Add("Number 1 is greater than number 2")
    Else
        lstOutput.Items.Add("Number 1 is smaller than number 2")
    End If
End Sub
```

2)

```
Private Sub btnStart_Click(...) Handles btnStart.Click
    Dim x As Integer = 12
    Dim y As Integer = 10
    Dim z As Integer = 8
    If x - y > y - z Then
        x = x + y + z
        y = x - y
        z = x - z
    Else
        If x - y < y - z Then
            x = x + y - z
            y = x + y
            z = y + z
        Else
            x = x - y - z
            y = x + y
            z = z - y
        End If
    End If
    lstOutput.Items.Add("x = " & CStr(x))
    lstOutput.Items.Add("y = " & CStr(y))
    lstOutput.Items.Add("z = " & CStr(z))
End Sub
```

3) Suppose text box txtNum1 contains "8" and text box txtNum2 contains "10"

```
Private Sub btnStart_Click(...) Handles btnStart.Click
    Dim num1 As Integer = CInt(txtNum1.Text)
    Dim num2 As Integer = CInt(txtNum2.Text)
    num1 += num1 * 3
    num2 += num2 * 2
    If num1 = num2 Then
        lstOutput.Items.Add(CInt(num1) & " is equal to " & CInt(num2))
    Else
        If num1 > num2 Then
```

```

        lstOutput.Items.Add(CInt(num1) & " is greater than " & CInt(num2))
    Else
        lstOutput.Items.Add(CInt(num1) & " is smaller than " & CInt(num2))
    End If
End If
End Sub

```

4) Suppose you enter "80000" in text box txtIncome.

```

Private Sub btnStart_Click(...) Handles btnStart.Click
    Dim grossIncome As Single = CSng(txtIncome.Text)
    Dim deductible As Single = 12200
    Dim taxBase As Single = grossIncome - deductible
    If taxBase < 0 Then taxBase = 0

    Dim tax As Single = 0
    Select Case taxBase
        Case Is <= 18150
            tax = taxBase * 0.1
        Case Is <= 73800
            tax = taxBase * 0.15
        Case Is <= 148850
            tax = taxBase * 0.25
        Case Is <= 226850
            tax = taxBase * 0.28
        Case Is <= 405100
            tax = taxBase * 0.33
        Case Is <= 457600
            tax = taxBase * 0.35
        Case Else
            tax = taxBase * 0.396
    End Select
    lstOutput.Items.Add("Your tax is: " & FormatCurrency(tax))
End Sub

```

5)

```

Private Sub btnStart_Click(...) Handles btnStart.Click
    Dim year As Integer
    For year = 2000 To 2010 Step 3
        lstOutput.Items.Add("Year " & CStr(year))
    Next
    lstOutput.Items.Add("Current value of variable year is " & CStr(year))
End Sub

```

6)

```

Private Sub btnStart_Click(...) Handles btnStart.Click
    Dim x As Integer = 0
    Dim y As Integer = 0
    For i As Integer = 0 To 9
        y += i ^ x
        lstOutput.Items.Add(y)
    Next
End Sub

```

Zero exponent rule: any number with an exponent of zero is 1.

7)

```

Private Sub btnStart_Click(...) Handles btnStart.Click
    Dim num As Integer = 10

```

```

        Do While num > 0
            lstOutput.Items.Add(num)
            num -= 2
        Loop
    End Sub

```

8)

```

Private Sub btnStart_Click(...) Handles btnStart.Click
    Dim num As Integer = 1
    Do
        lstOutput.Items.Add(num)
        num += 1
    Loop Until num >= 5
End Sub

```

- 9) A text box txtGrade allows the user to enter a student's grade (1 to 12). It calculates the average amount of time needed for homework each day. Suppose you enter "8" in the box, what will be displayed in the list box?

```

Private Sub btnStart_Click(...) Handles btnStart.Click
    Dim hours As Integer = 0, minutes As Integer = 0
    Dim grade As Integer = CInt(txtNum1.Text)
    For i As Integer = 1 To grade
        minutes += 20
    Next

    If minutes >= 60 Then
        hours = Int(minutes / 60)
        minutes = minutes Mod 60
    End If

    Dim strTime As String = ""
    If hours > 0 Then
        If hours = 1 Then
            strTime = CStr(hours) & " hour"
        Else
            strTime = CStr(hours) & " hours"
        End If
        If minutes > 0 Then
            strTime &= " " & CStr(minutes) & " minutes"
        End If
    Else
        strTime = CStr(minutes) & " minutes"
    End If
    lstOutput.Items.Add("Daily homework time is " & strTime)
End Sub

```

10)

```

Private Sub btnStart_Click(...) Handles btnStart.Click
    For Year As Integer = 2010 To 2013 Step 2
        For mon = 3 To 12 Step 3
            Select Case mon
                Case 1
                    strMon = "January"
                Case 2
                    strMon = "February"
                Case 3

```

```

        strMon = "March"
    Case 4
        strMon = "April"
    Case 5
        strMon = "May"
    Case 6
        strMon = "June"
    Case 7
        strMon = "July"
    Case 8
        strMon = "August"
    Case 9
        strMon = "September"
    Case 10
        strMon = "October"
    Case 11
        strMon = "November"
    Case 12
        strMon = "December"
End Select
lstOutput.Items.Add(strMon & ", " & CStr(Year))
Next
Next
End Sub

```

4. Create a VB project named "WaterBill" to calculate single-family domestic customer's water bills for a city. The rules are:
 - Base fee: \$38.66
 - First 14 hundred cubic feet (HCF) used are billed at \$3.612 per HCF
 - Second 14 HCF used are billed at \$3.917 per HCF
 - Each HCF used after the initial 28 HCF is billed at \$4.398 per HCF

Your project should design a GUI that allows the user to enter the volume of water in HCF consumed by a household, and it should report the result in a read-only text box.
5. Create a VB program to calculate the sum of $1 + 2 + 3 + \dots + 36$. Your program should use a list box to display the result of each step.
6. Suppose the population growth rate of fast growing city is 10% each year. In 2010, the population was 10,000. If the growth rate keeps constant for an enough long period of time, in what year will the population be doubled? Create a VB project to calculate the population growth. Your application GUI should allow the user to enter any initial population and any growth rate, and use a list box to display the predicted population of each year.